Chad Galloway
CST-250 Programming in C# II
Grand Canyon University
Nov. 23, 2025
Milestone 5

Files
https://github.com/CGalloway3/CST-250-Projects/tree/master/Milestone5

Video
https://www.loom.com/share/835bf81023794a3e98b0df8fe8b32b02

FLOW CHART

```
/*
* Chad Galloway
* CST - 250 Programming in C# II
* 11/23/2025
* Mine Sweeper GUI App
* Milestone 5
* References:
*/
```

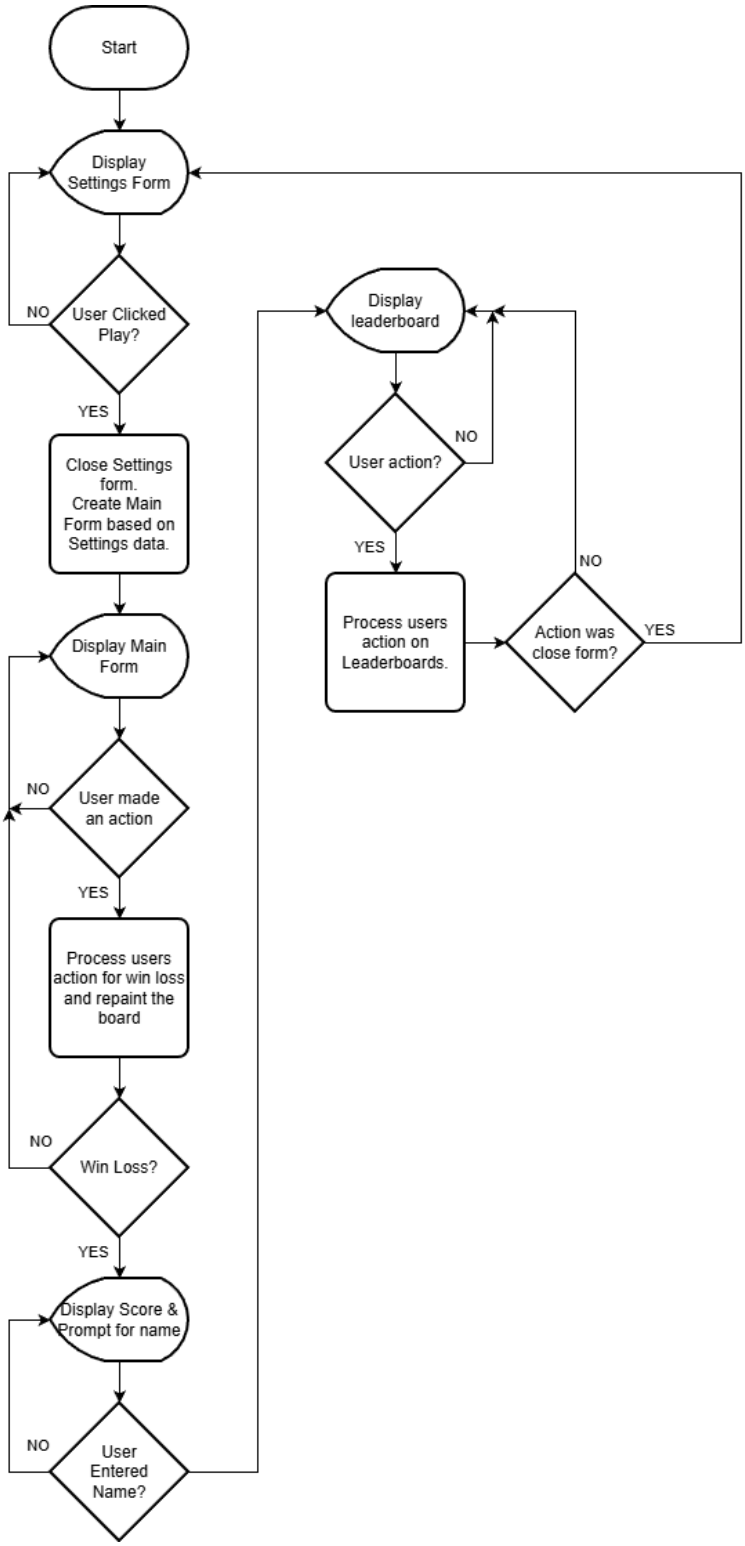


Figure `: Flow chart of milestone 5

UML Class Diagram

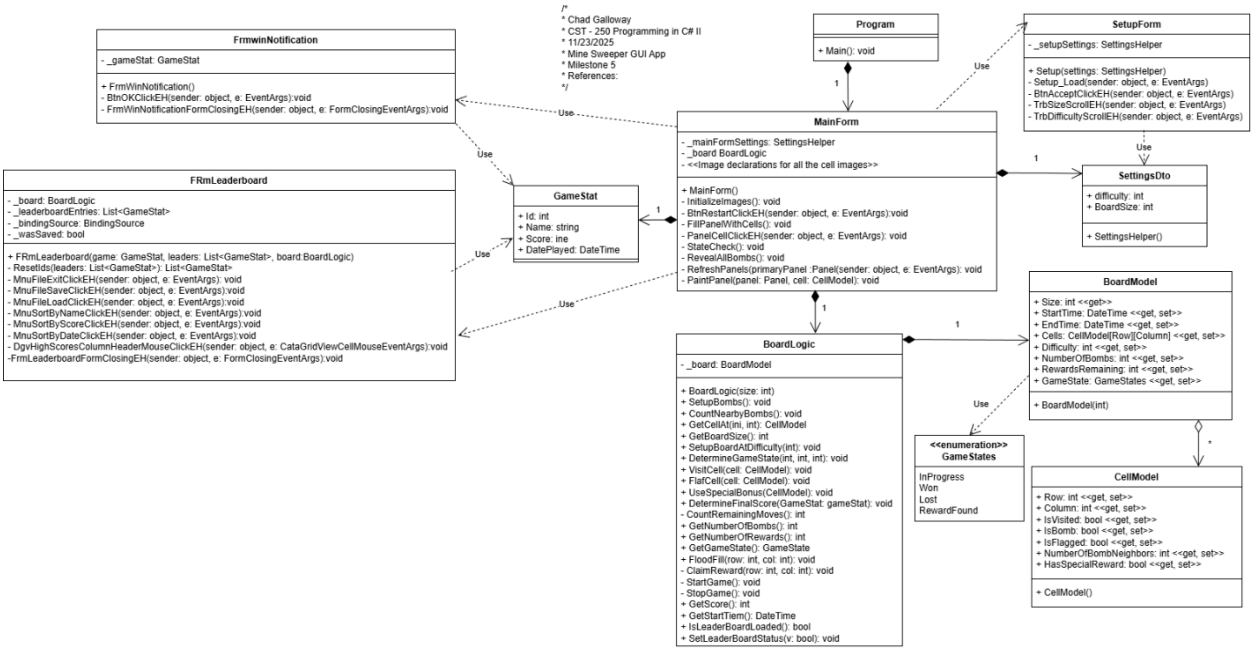


Figure 2: UML Class Diagram of milestone 5

Screen Shots

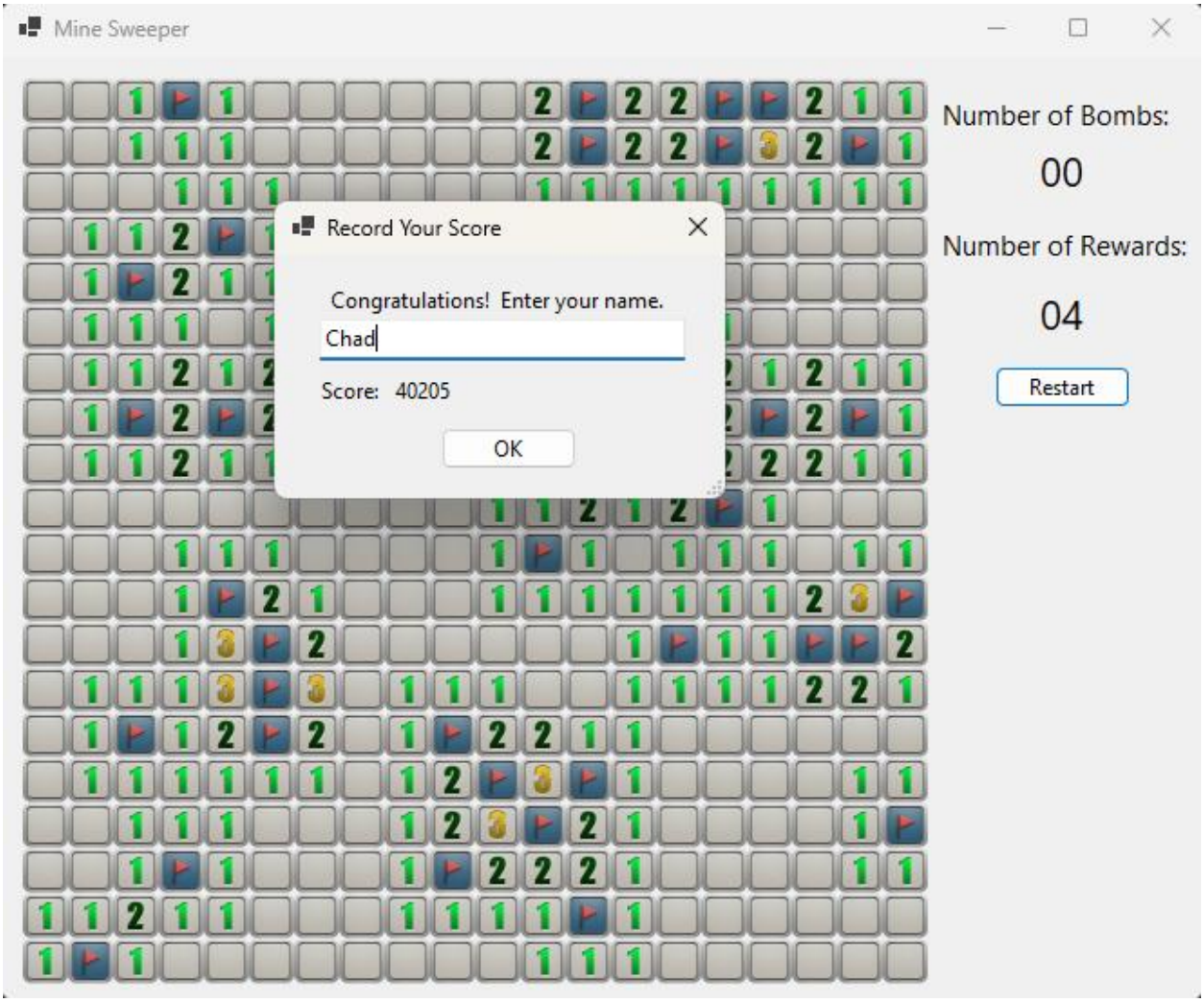


Figure 3: Screenshot of a Win notification displayed to the user.

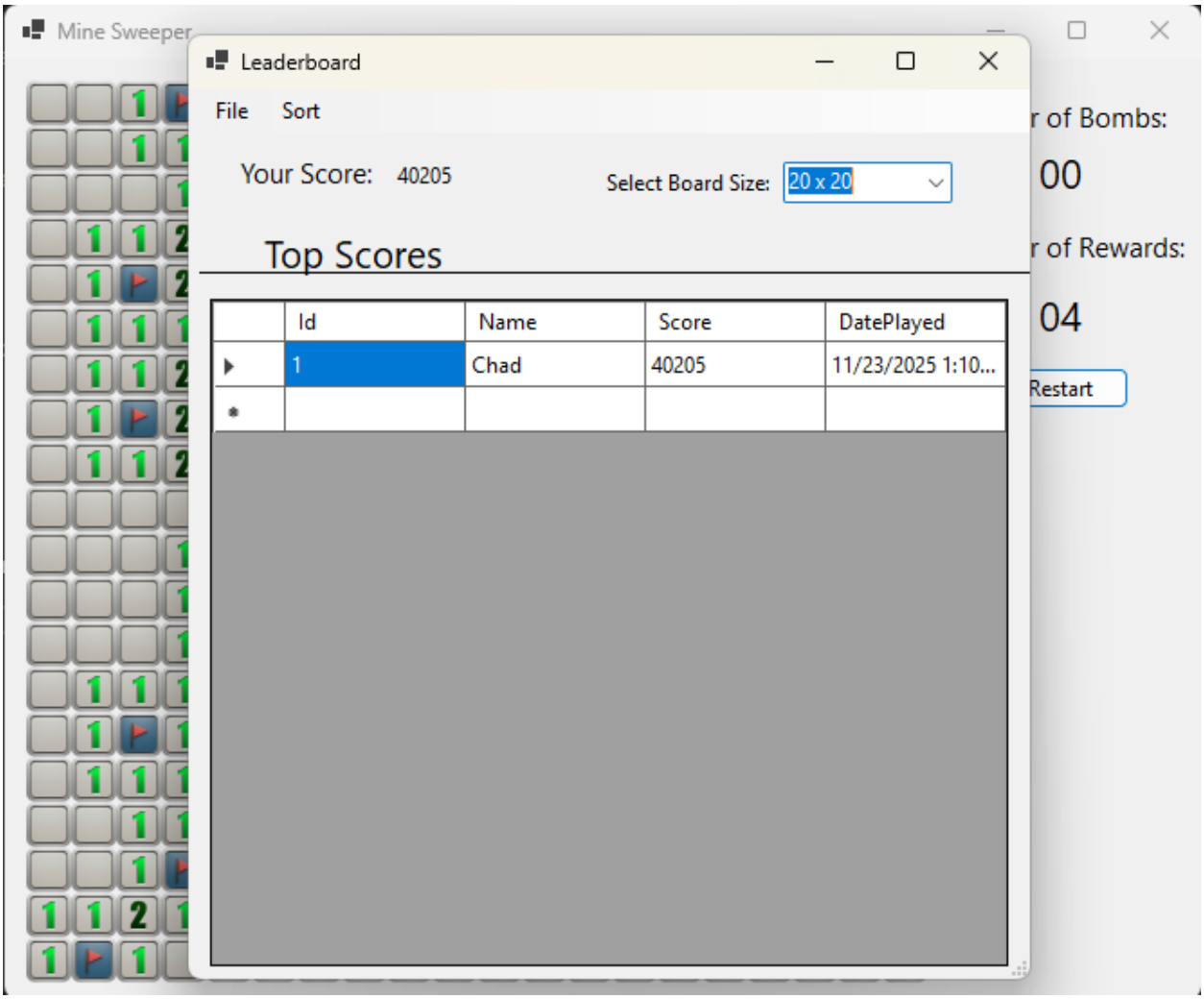


Figure 4: Screenshot of the leaderboard being presented to the user.

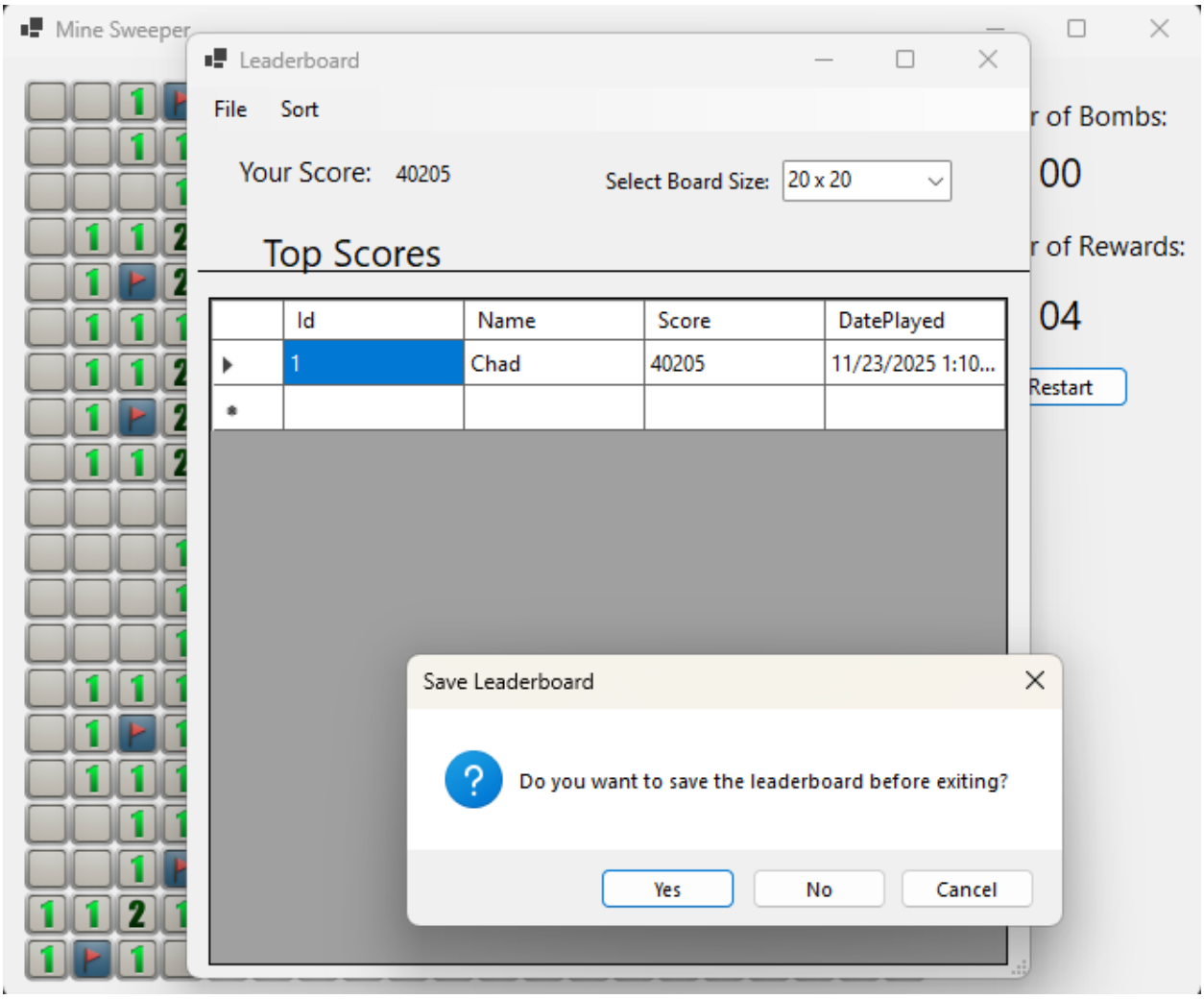


Figure 5: Screenshot of save notification warning about exiting the form before saving the newest leaderboard addition.

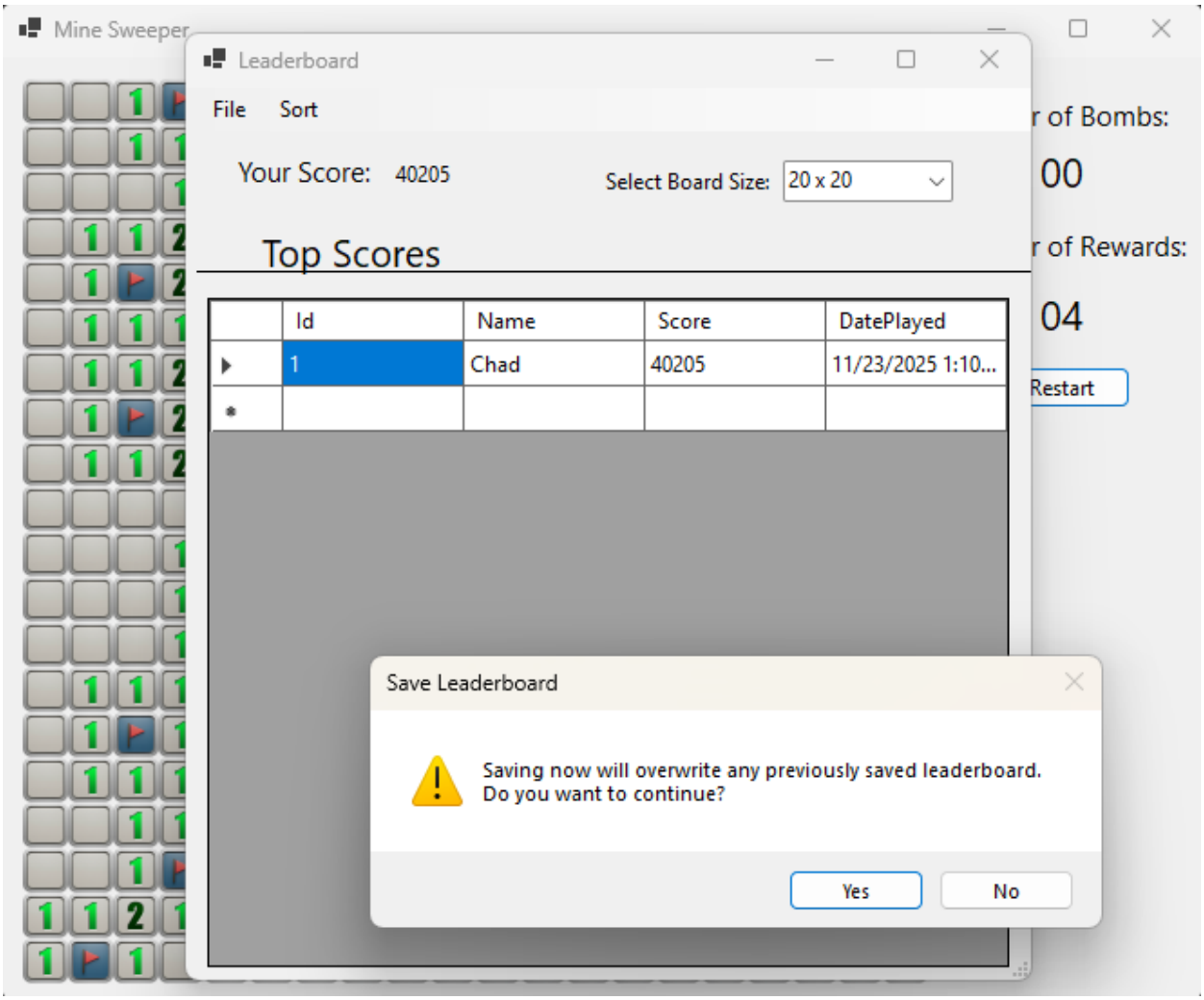


Figure 6: Screenshot of an overwrite warning when the leaderboard is not loaded yet. If the user loads the leaderboard in that is remembered by the application and they won't receive this notification again. Currently if the player selects save before exit but do not overwrite the form just closes without doing anything. I simply ran out of time to correct this action. Eventually, the application should return to the Leaderboard form so the user can load the leaderboard and try again to save. The save and load actions are also both coded into the click event handlers and ideally, they should be in another method at minimum or better another class for a data object but no more time.

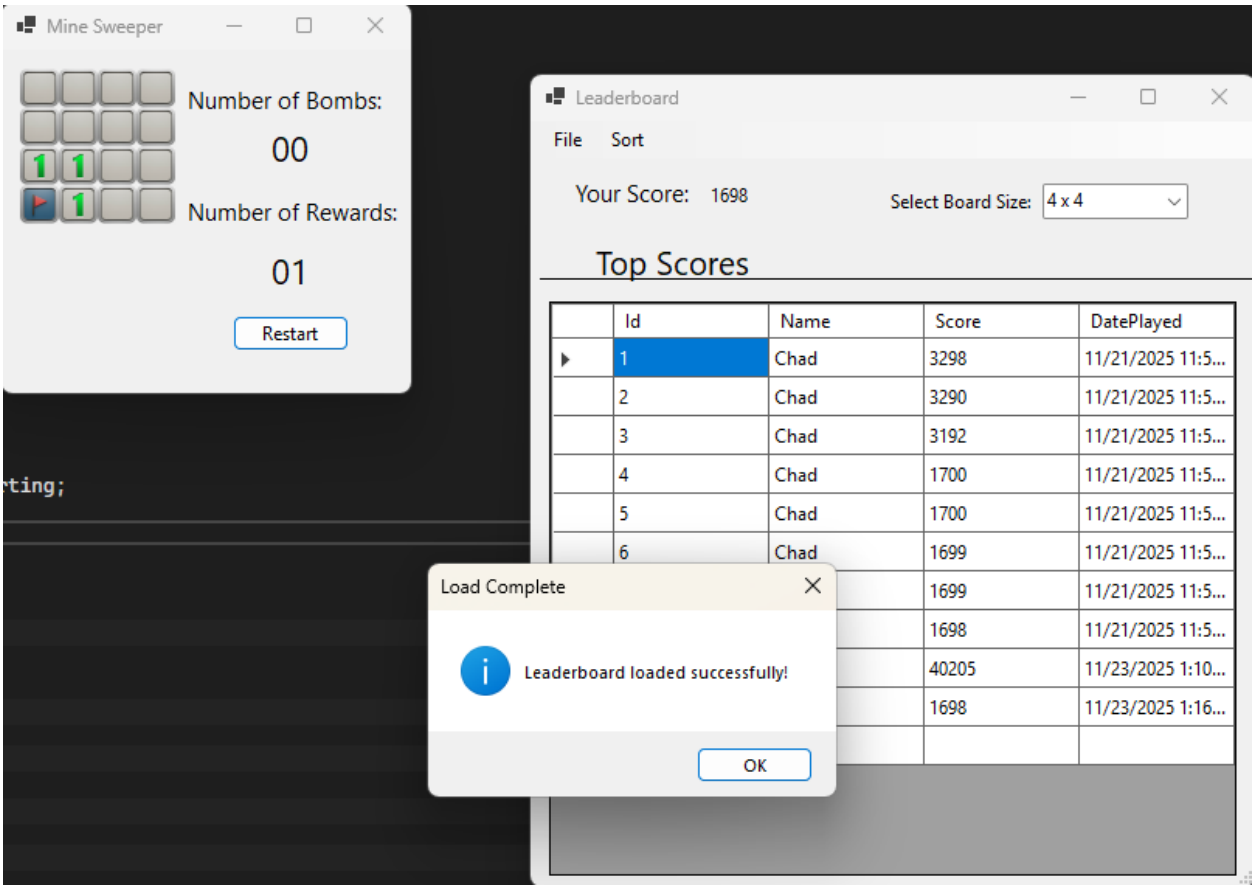


Figure 7: Screenshot of the leaderboard successfully loading in to the player. The load logic occurs in the click event handler. I know this is not ideal and not proper separation of concerns but I ran out of time to try and refactor. Just like last week with the message boxes in the board logic which are now fixed. I will look into fixing this behavior next week. Ideally what is in my head is a leader board is just a list so I want to look into extending the List<GameStat> object for a leaderboard class and add in the save and load actions there.

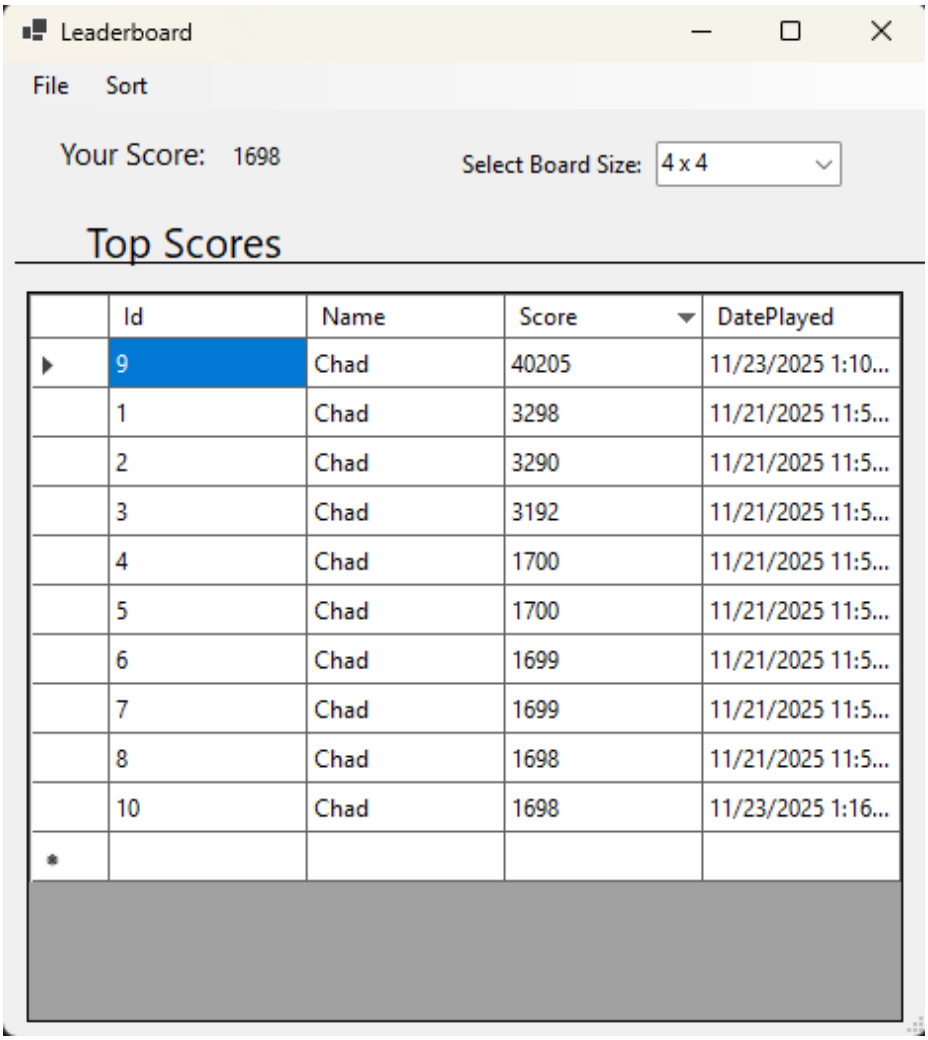


Figure 8: Screenshot of the leaderboard being sorted by the user by score. Both the menu commands and the dgv header clicks call the same method which flip flops the header sorting order. To get around the difficulty of sorting a list with the dgv default sort methods (it is not the correct IEnumerable sortable list type) I removed the list from the dgv, sorted it, and then readded it to the dgv.
games player data.

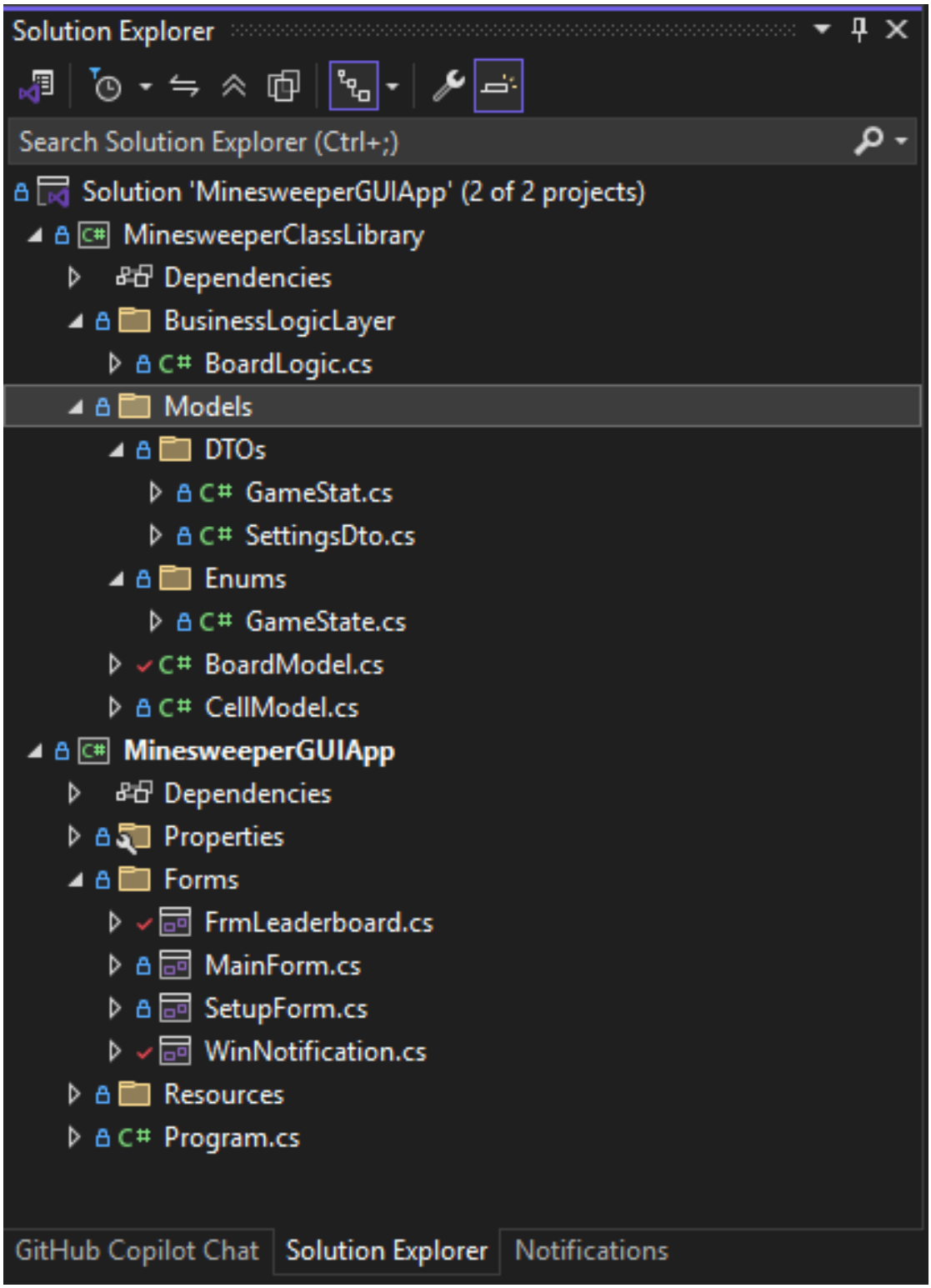
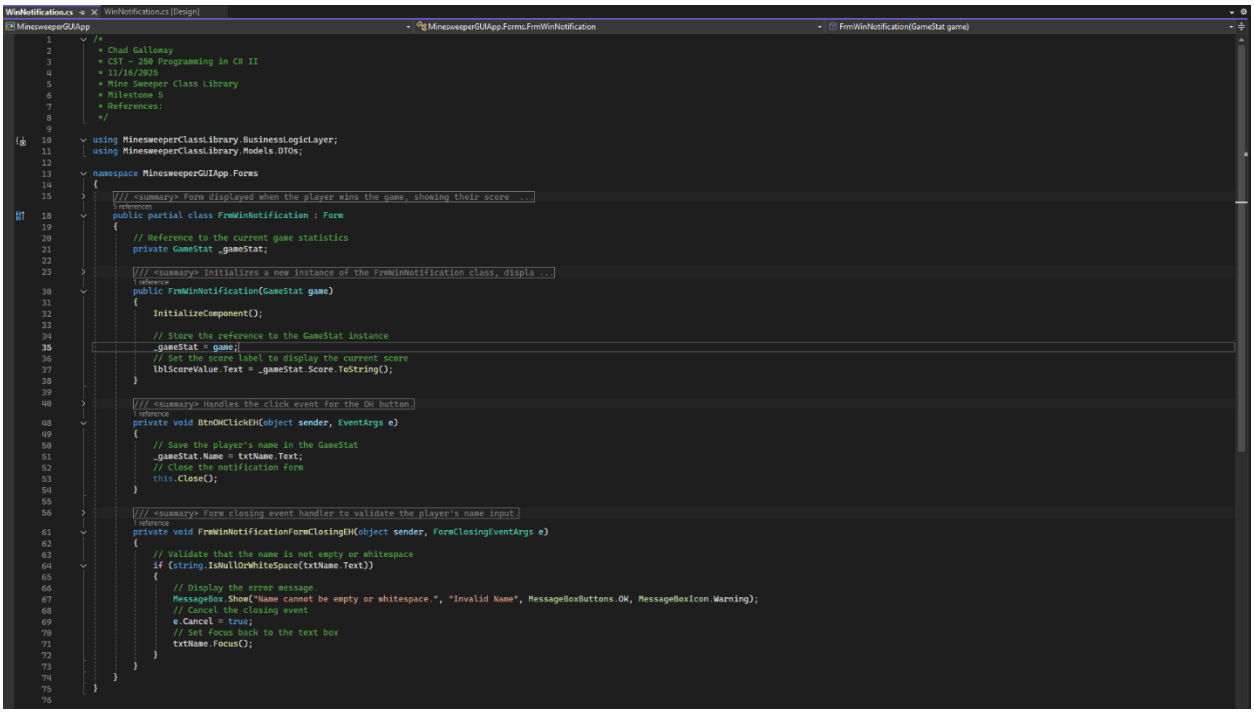
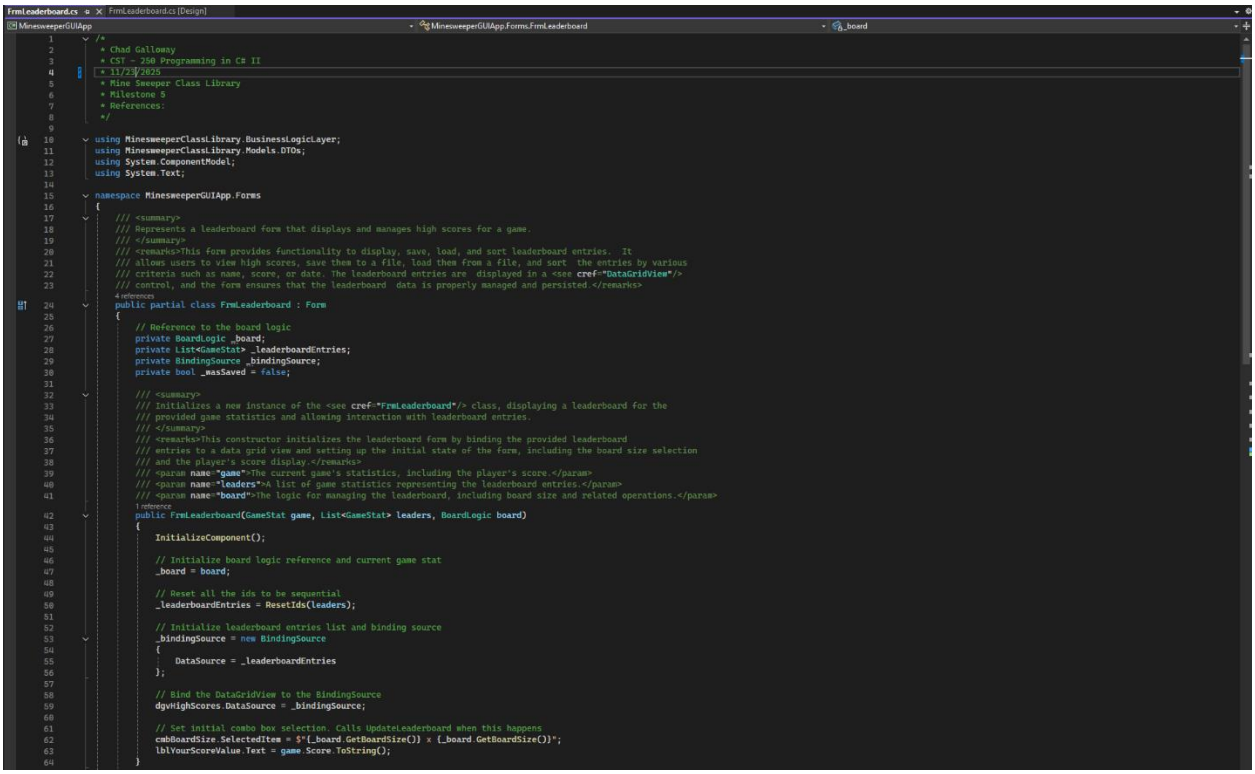


Figure 9: Screenshot of the Solution Explorer for the project as it stands. The required weeks classes have been added in.



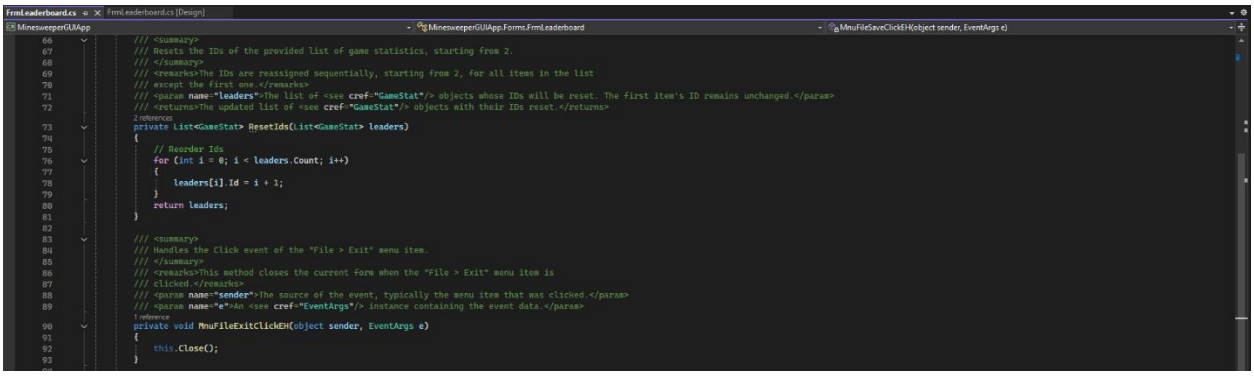
```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/16/2025
 * Mine Sweeper Class Library
 * Milestone 5
 * References:
 */

using MinesweeperClassLibrary.BusinessLogicLayer;
using MinesweeperClassLibrary.Models.DTOs;

namespace MinesweeperGUIApp.Forms
{
    /// <summary> Form displayed when the player wins the game, showing their score ...
    public partial class FrmWinNotification : Form
    {
        // Reference to the current game statistics
        private GameStat _gameStat;

        /// <summary> Initializes a new instance of the FrmWinNotification class, displa ...
        public FrmWinNotification(GameStat game)
        {
            InitializeComponent();

            // Store the reference to the GameStat instance
            _gameStat = game;
            // Set the score label to display the current score
            lblScoreValue.Text = _gameStat.Score.ToString();
        }

        /// <summary> Handles the click event for the OK button.
        private void BtnOKClickEH(object sender, EventArgs e)
        {
            // Save the player's name in the GameStat
            _gameStat.Name = txtName.Text;
            // Close the notification form
            this.Close();
        }

        /// <summary> Form closing event handler to validate the player's name input.
        private void FrmWinNotificationFormClosingEH(object sender, FormClosingEventArgs e)
        {
            // Validate that the name is not empty or whitespace
            if (string.IsNullOrWhiteSpace(txtName.Text))
            {
                // Display the error message.
                MessageBox.Show("Name cannot be empty or whitespace.", "Invalid Name", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                // Cancel the closing event
                e.Cancel = true;
                // Set focus back to the text box
                txtName.Focus();
            }
        }
    }
}
```

Figure 10: Screenshot of the code behind the win notification form. The summaries are collapsed for space. This is a simple form with the responsibility of recording the players name in the game stat object when they win a game. When it is initialized, it accepts the game stat object a parameter. The form will not close until the user enters a proper name. Names with a comma in them are acceptable by the form but when that happens it causes issues with the leaderboard loading. This needs to be fixed but again I ran out of time. I spent a majority of the week coding my own leaderboard logic and opened the milestone document late Thursday to find out I was not following the exact implementation for the leaderboard that was laid out there and I had to throw away most of what I had already done. Some remnants of that can still be seem, i.e. the combobox on the leaderboard form. It was to allow the user to view the leaderboard by the sizes so they compared their scores to those of players in the same board size. This late game shift put me way behind schedule for the week and I have been playing catch up ever since. I hope to fix all these small issues next week.



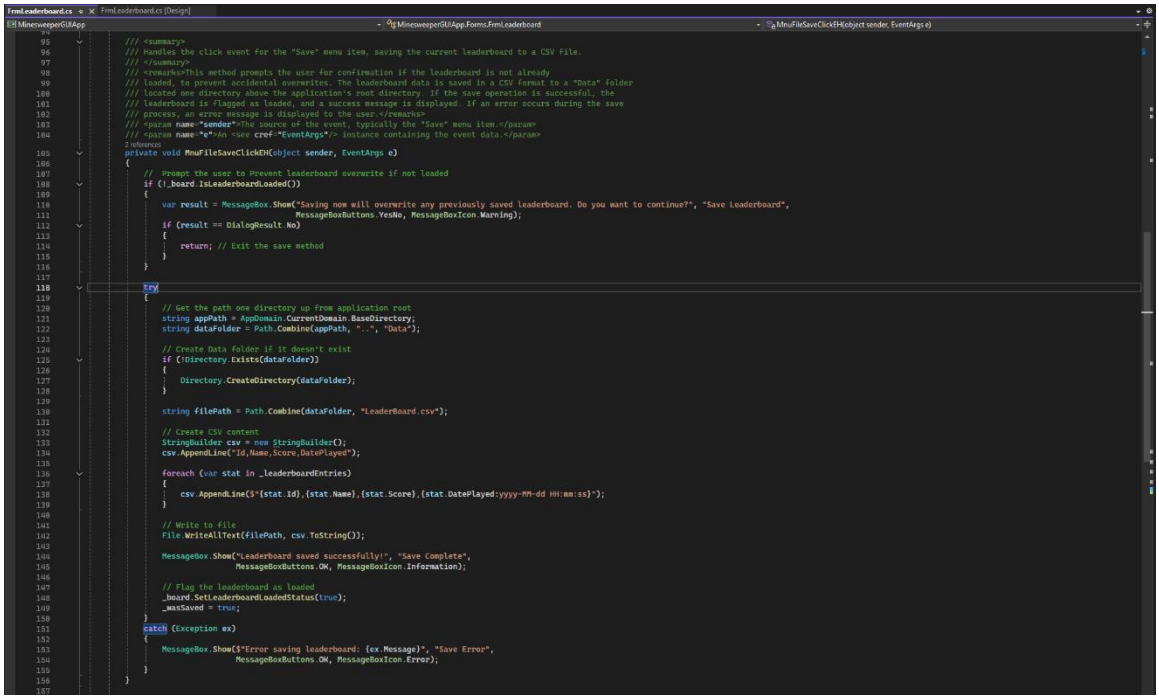
```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/23/2025
 * Mine Sweeper Class Library
 * Milestone 5
 * References:
 */

using MinesweeperClassLibrary.BusinessLogicLayer;
using MinesweeperClassLibrary.Models.DTOs;
using System.ComponentModel;
using System.Text;

namespace MinesweeperGUIApp.Forms
{
    /// <summary>
    /// Represents a leaderboard form that displays and manages high scores for a game.
    /// </summary>
    /// <remarks>This form provides functionality to display, save, load, and sort leaderboard entries.  It
    /// allows users to view high scores, save them to a file, load them from a file, and sort  the entries by various
    /// criteria such as name, score, or date. The leaderboard entries are  displayed in a <see cref="DataGridView"/>
    /// control, and the form ensures that the leaderboard  data is properly managed and persisted.</remarks>
    4 references
    public partial class FrmLeaderboard : Form
    {
        // Reference to the board logic
        private BoardLogic _board;
        private List<GameStat> _leaderboardEntries;
        private BindingSource _bindingSource;
        private bool _wasSaved = false;

        /// <summary>
        /// Initializes a new instance of the <see cref="FrmLeaderboard"/> class, displaying a leaderboard for the
        /// provided game statistics and allowing interaction with leaderboard entries.
        /// </summary>
        /// <remarks>This constructor initializes the leaderboard form by binding the provided leaderboard
        /// entries to a data grid view and setting up the initial state of the form, including the board size selection
        /// and the player's score display.</remarks>
        /// <param name="game">The current game's statistics, including the player's score.</param>
        /// <param name="leaders">A list of game statistics representing the leaderboard entries.</param>
        /// <param name="board">The logic for managing the leaderboard, including board size and related operations.</param>
        1 reference
        public FrmLeaderboard(GameStat game, List<GameStat> leaders, BoardLogic board)
        {
            InitializeComponent();

            // Initialize board logic reference and current game stat
            _board = board;

            // Reset all the ids to be sequential
            _leaderboardEntries = ResetIds(leaders);

            // Initialize leaderboard entries list and binding source
            _bindingSource = new BindingSource
            {
                DataSource = _leaderboardEntries
            };

            // Bind the DataGridView to the BindingSource
            dgvHighScores.DataSource = _bindingSource;

            // Set initial combo box selection. Calls UpdateLeaderboard when this happens
            cmbBoardSize.SelectedItem = $"{_board.GetBoardSize()} x {_board.GetBoardSize()}";
            lblYourScoreValue.Text = game.Score.ToString();
        }
```

Figure 11: Screenshot of the code behind the leaderboard form. This is the citations and constructor. See Figure 12 - 15 for the methods.



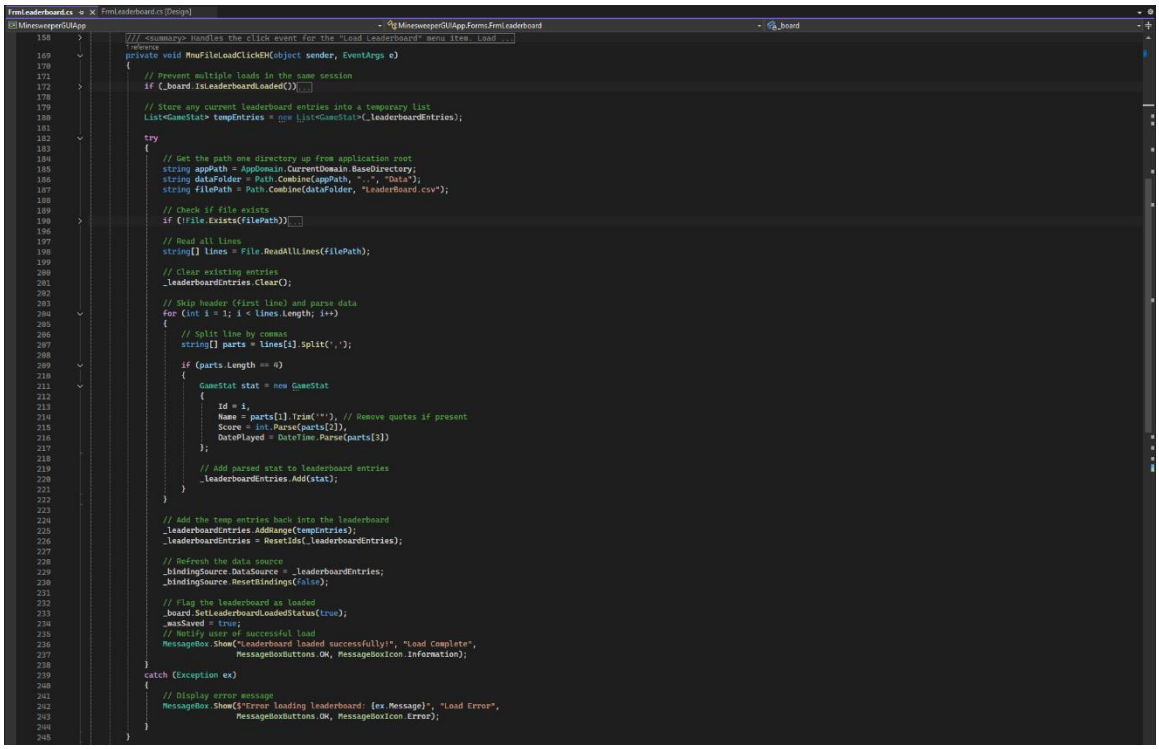
```
        /// <summary>
        /// Resets the IDs of the provided list of game statistics, starting from 2.
        /// </summary>
        /// <remarks>The IDs are reassigned sequentially, starting from 2, for all items in the list
        /// except the first one.</remarks>
        /// <param name="leaders">The list of <see cref="GameStat"/> objects whose IDs will be reset. The first item's ID remains unchanged.</param>
        /// <returns>The updated list of <see cref="GameStat"/> objects with their IDs reset.</returns>
        2 references
        private List<GameStat> ResetIds(List<GameStat> leaders)
        {
            // Reorder Ids
            for (int i = 0; i < leaders.Count; i++)
            {
                leaders[i].Id = i + 1;
            }
            return leaders;
        }

        /// <summary>
        /// Handles the Click event of the "File > Exit" menu item.
        /// </summary>
        /// <remarks>This method closes the current form when the "File > Exit" menu item is
        /// clicked.</remarks>
        /// <param name="sender">The source of the event, typically the menu item that was clicked.</param>
        /// <param name="e">An <see cref="EventArgs"/> instance containing the event data.</param>
        1 reference
        private void MnuFileExitClickEH(object sender, EventArgs e)
        {
            this.Close();
        }
```

Figure 12: Screenshot of the reset ids and file -> exit click event handler for the leaderboard form



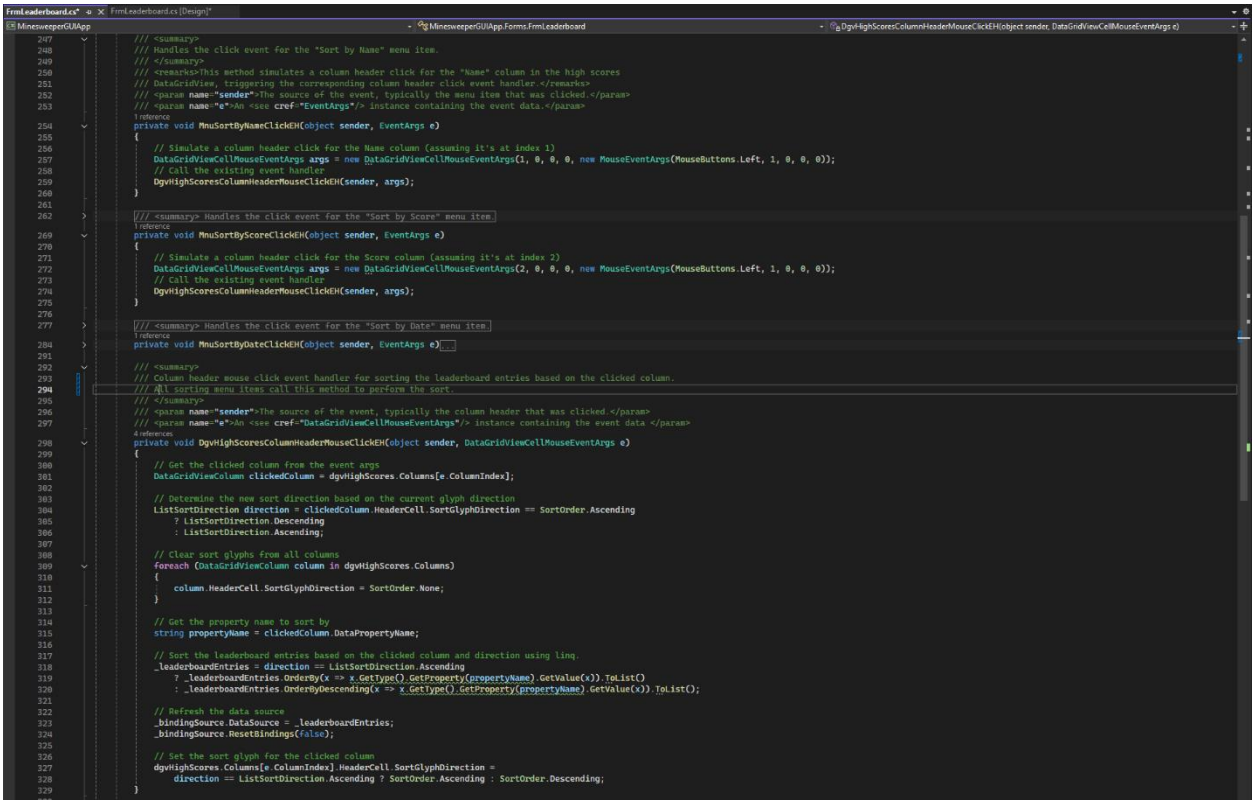
```
/// <summary>
/// Handles the click event for the "Save" menu item, saving the current leaderboard to a CSV file.
/// </summary>
/// <remarks>This method prompts the user for confirmation if the leaderboard is not already
/// loaded, to prevent accidental overwrites. The leaderboard data is saved in a CSV format to a "Data" folder
/// located one directory above the application's root directory. If the save operation is successful, the
/// leaderboard is flagged as loaded, and a success message is displayed. If an error occurs during the save
/// process, an error message is displayed to the user.</remarks>
/// <param name="sender">The source of the event, typically the "Save" menu item.</param>
/// <param name="e">An <see cref="EventArgs"/> instance containing the event data.</param>
2 references
private void MnuFileSaveClickEH(object sender, EventArgs e)
{
    //  Prompt the user to Prevent leaderboard overwrite if not loaded
    if (!_board.IsLeaderboardLoaded())
    {
        var result = MessageBox.Show("Saving now will overwrite any previously saved leaderboard. Do you want to continue?", "Save Leaderboard",
                                     MessageBoxButtons.YesNo, MessageBoxIcon.Warning);

        if (result == DialogResult.No)
        {
            return; // Exit the save method
        }
    }

    try
    {
        // Get the path one directory up from application root
        string appPath = AppDomain.CurrentDomain.BaseDirectory;
        string dataFolder = Path.Combine(appPath, "..", "Data");

        // Create Data folder if it doesn't exist
        if (!Directory.Exists(dataFolder))
        {
            Directory.CreateDirectory(dataFolder);
        }

        string filePath = Path.Combine(dataFolder, "LeaderBoard.csv");

        // Create CSV content
        StringBuilder csv = new StringBuilder();
        csv.AppendLine("Id,Name,Score,DatePlayed");

        foreach (var stat in _leaderboardEntries)
        {
            csv.AppendLine($"{stat.Id},{stat.Name},{stat.Score},{stat.DatePlayed:yyyy-MM-dd HH:mm:ss}");
        }

        // Write to file
        File.WriteAllText(filePath, csv.ToString());

        MessageBox.Show("Leaderboard saved successfully!", "Save Complete",
                        MessageBoxButtons.OK, MessageBoxIcon.Information);

        // Flag the leaderboard as loaded
        _board.SetLeaderboardLoadedStatus(true);
        _wasSaved = true;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error saving leaderboard: {ex.Message}", "Save Error",
                        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Figure 13: Screenshot of the file -> save click event handler. As noted above the logic to handle the save is in this method as well as the presentation. I know this is not ideal but I was running out of time to complete the project. I will refactor next week.



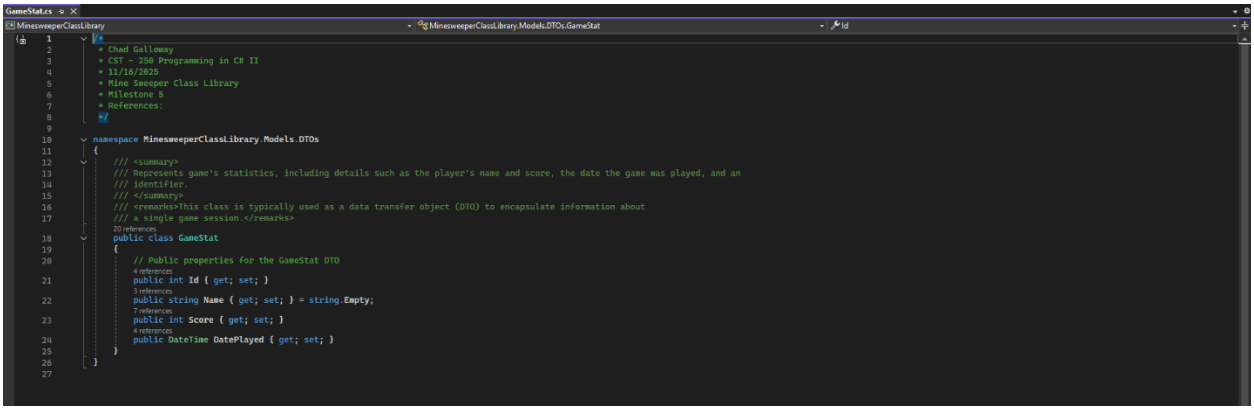
```
/// <summary> Handles the click event for the "Load Leaderboard" menu item. Load ...
1 reference
private void MnuFileLoadClickEH(object sender, EventArgs e)
{
    // Prevent multiple loads in the same session
    if (_board.IsLeaderboardLoaded())...

    // Store any current leaderboard entries into a temporary list
    List<GameStat> tempEntries = new List<GameStat>(_leaderboardEntries);

    try
    {
        // Get the path one directory up from application root
        string appPath = AppDomain.CurrentDomain.BaseDirectory;
        string dataFolder = Path.Combine(appPath, "..", "Data");
        string filePath = Path.Combine(dataFolder, "LeaderBoard.csv");

        // Check if file exists
        if (!File.Exists(filePath))...

        // Read all lines
        string[] lines = File.ReadAllLines(filePath);

        // Clear existing entries
        _leaderboardEntries.Clear();

        // Skip header (first line) and parse data
        for (int i = 1; i < lines.Length; i++)
        {
            // Split line by commas
            string[] parts = lines[i].Split(',');

            if (parts.Length == 4)
            {
                GameStat stat = new GameStat
                {
                    Id = i,
                    Name = parts[1].Trim('"'), // Remove quotes if present
                    Score = int.Parse(parts[2]),
                    DatePlayed = DateTime.Parse(parts[3])
                };

                // Add parsed stat to leaderboard entries
                _leaderboardEntries.Add(stat);
            }
        }

        // Add the temp entries back into the leaderboard
        _leaderboardEntries.AddRange(tempEntries);
        _leaderboardEntries = ResetIds(_leaderboardEntries);

        // Refresh the data source
        _bindingSource.DataSource = _leaderboardEntries;
        _bindingSource.ResetBindings(false);

        // Flag the leaderboard as loaded
        _board.SetLeaderboardLoadedStatus(true);
        _wasSaved = true;
        // Notify user of successful load
        MessageBox.Show("Leaderboard loaded successfully!", "Load Complete",
                        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        // Display error message
        MessageBox.Show($"Error loading leaderboard: {ex.Message}", "Load Error",
                        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Figure 14: Screenshot of the file -> load click event handler. If board has a leaderboard loaded and if file does not exist are collapsed for space they just display a message box and return.



```
/// <summary>
/// Handles the click event for the "Sort by Name" menu item.
/// </summary>
/// <remarks>This method simulates a column header click for the "Name" column in the high scores
/// DataGridView, triggering the corresponding column header click event handler.</remarks>
/// <param name="sender">The source of the event, typically the menu item that was clicked.</param>
/// <param name="e">An <see cref="EventArgs"/> instance containing the event data.</param>
1 reference
private void MnuSortByNameClickEH(object sender, EventArgs e)
{
    // Simulate a column header click for the Name column (assuming it's at index 1)
    DataGridViewCellMouseEventArgs args = new DataGridViewCellMouseEventArgs(1, 0, 0, 0, new MouseEventArgs(MouseButtons.Left, 1, 0, 0, 0));
    // Call the existing event handler
    DgvHighScoresColumnHeaderMouseClickEH(sender, args);
}

/// <summary> Handles the click event for the "Sort by Score" menu item.
1 reference
private void MnuSortByScoreClickEH(object sender, EventArgs e)
{
    // Simulate a column header click for the Score column (assuming it's at index 2)
    DataGridViewCellMouseEventArgs args = new DataGridViewCellMouseEventArgs(2, 0, 0, 0, new MouseEventArgs(MouseButtons.Left, 1, 0, 0, 0));
    // Call the existing event handler
    DgvHighScoresColumnHeaderMouseClickEH(sender, args);
}

/// <summary> Handles the click event for the "Sort by Date" menu item.
1 reference
private void MnuSortByDateClickEH(object sender, EventArgs e)...

/// <summary>
/// Column header mouse click event handler for sorting the leaderboard entries based on the clicked column.
/// All sorting menu items call this method to perform the sort.
/// </summary>
/// <param name="sender">The source of the event, typically the column header that was clicked.</param>
/// <param name="e">An <see cref="DataGridViewCellMouseEventArgs"/> instance containing the event data </param>
4 references
private void DgvHighScoresColumnHeaderMouseClickEH(object sender, DataGridViewCellMouseEventArgs e)
{
    // Get the clicked column from the event args
    DataGridViewColumn clickedColumn = dgvHighScores.Columns[e.ColumnIndex];

    // Determine the new sort direction based on the current glyph direction
    ListSortDirection direction = clickedColumn.HeaderCell.SortGlyphDirection == SortOrder.Ascending
        ? ListSortDirection.Descending
        : ListSortDirection.Ascending;

    // Clear sort glyphs from all columns
    foreach (DataGridViewColumn column in dgvHighScores.Columns)
    {
        column.HeaderCell.SortGlyphDirection = SortOrder.None;
    }

    // Get the property name to sort by
    string propertyName = clickedColumn.DataPropertyName;

    // Sort the leaderboard entries based on the clicked column and direction using linq.
    _leaderboardEntries = direction == ListSortDirection.Ascending
        ? _leaderboardEntries.OrderBy(x => x.GetType().GetProperty(propertyName).GetValue(x)).ToList()
        : _leaderboardEntries.OrderByDescending(x => x.GetType().GetProperty(propertyName).GetValue(x)).ToList();

    // Refresh the data source
    _bindingSource.DataSource = _leaderboardEntries;
    _bindingSource.ResetBindings(false);

    // Set the sort glyph for the clicked column
    dgvHighScores.Columns[e.ColumnIndex].HeaderCell.SortGlyphDirection =
        direction == ListSortDirection.Ascending ? SortOrder.Ascending : SortOrder.Descending;
}
```

Figure 15: Screenshot of all the sorting logic methods. The main meat of action here is the dgv column header click event. It gets the column clicked, figures out what direction it is currently sorted in, clears all the columns of their sorting arrow, records the name of the clicked column, and then uses LinQ to sort the column by either ascending or descending and push that sorted list back to the original leaderboard. After the list is updated the binding source is refreshed and the sorting marker is put back at the top of the correct column showing the correct direction. Each of the sort menu items just sets a valid dgv event arg for the specific column and calls this method as if the column headers were clicked. Doing this eliminates the need to do logic in each menu items click event handler and passes that responsibility along to a method that needs to handle the logic also.



```
/*
 * Chad Galloway
 * CST - 250 Programming in C# II
 * 11/16/2025
 * Mine Sweeper Class Library
 * Milestone 5
 * References:
 */

namespace MinesweeperClassLibrary.Models.DTOs
{
    /// <summary>
    /// Represents game's statistics, including details such as the player's name and score, the date the game was played, and an
    /// identifier.
    /// </summary>
    /// <remarks>This class is typically used as a data transfer object (DTO) to encapsulate information about
    /// a single game session.</remarks>
    20 references
    public class GameStat
    {
        // Public properties for the GameStat DTO
        4 references
        public int Id { get; set; }
        3 references
        public string Name { get; set; } = string.Empty;
        7 references
        public int Score { get; set; }
        4 references
        public DateTime DatePlayed { get; set; }
    }
}
```

Figure 16: Screenshot of the game stat class that holds all the data for the currently running games player data.

This weeks milestone demonstrates the implementation of data persistence and management in a WinForms application through file I/O operations and dynamic data visualization. The key concepts include creating a GameStat model class to encapsulate player statistics with properties for identification, scoring, and timestamps, implementing a multi-form architecture to capture and display leaderboard data, and utilizing the DataGridView control for presenting tabular information to users. The milestone showcases file handling through Save and Load menu operations that write to and read from CSV text files, enabling persistent storage of game statistics across application sessions. Additionally, the implementation of sorting algorithms (by name, score, and date) demonstrates data manipulation and LINQ operations for organizing collections. This milestone reinforces the separation of concerns by managing data access operations independently from the UI layer, while the menu system implementation illustrates event-driven programming patterns common in Windows desktop applications. The integration of these features provides players with a comprehensive scoring system that tracks performance over time and enables competitive gameplay through persistent leaderboard functionality.